Fintech Architecture — Dewayne Ballard

To: Dewayne Ballard **From:** Eric Beser

Here is a document answering the questions of how I would implement this application.

---

1. Multi-tenant table structure

The tenant boundary is an **organization** entity with a UUID primary key. Every tenant-owned table carries an `org_id` column as a foreign key back to that root, and that column is the only thing RLS policies key on.

Identity is deliberately decoupled from tenancy. One user row per human, keyed to the Supabase Auth UUID. A separate junction table — memberships — joins users to organizations with a role (owner, admin, loan_officer, viewer) and a status (invited, active, removed). A user who works for two brokerages has two membership rows, not two user rows. When someone leaves an organization, their membership status flips to removed — the user row is never deleted, because their signature must remain on historical records.

The property, loan, and payment tables all hang off org_id directly. Properties carry address data and an appraised-value column (always stored as integer cents — never a floating-point number or SQL numeric used for display). Loans carry principal, interest rate (stored in basis points, integer), term length, origination date, amortization type, status, and a denormalized current_balance_cents that is a cached snapshot of the ledger. Payments carry the borrower-facing event data: amount, method, received timestamp, allocation columns for principal/interest/fee/escrow, Stripe payment intent ID, and status.

The critical separation is between payments and the underlying ledger. A payment is a business event that may be pending, clearing, cleared, reversed, or refunded. The ledger is an append-only record of every money movement that actually happened, with one row per posting. A single payment often produces four ledger entries (fee first, then interest, then principal, then escrow). Keeping these two concepts in separate tables is what allows ACH reversals five days after the fact to be handled without rewriting history.

---

2. RLS for strict tenant isolation

Supabase Auth hands every client a JWT. Out of the box that token carries the user's Auth UUID — it does not know which organization the user is acting as, which matters when users belong to more than one org.

The solution is a custom access token hook that runs at login and on every refresh. It looks up the user's active organization (either explicitly chosen and persisted to the user's preferences, or defaulted to the first active membership) and injects an `org_id` claim into the JWT. A helper function reads that claim and is referenced by every RLS policy in the database.

Every tenant table has three policies minimum: SELECT, INSERT, UPDATE. DELETE is almost never granted — deletion happens via UPDATE of a deleted_at column, which preserves audit trail. Each policy checks two things: that the row's org_id matches the token's org_id claim, AND that an active membership row exists in that org for the requesting user. The membership check is the important one — it means a user who was removed from the org but whose JWT has not expired yet still loses access on their next query, not up to an hour later when the token rolls over.

The service_role key bypasses RLS entirely. That key is used by the Stripe webhook handler, background jobs, admin operations, and nothing else. Every code path that uses it must manually scope every query by org_id in SQL, because the database is not going to help. The service_role key never leaves the backend.

Foreign keys are not a substitute for RLS. FKs guarantee referential integrity — that a loan's property_id points to a real property. They do not prevent a malicious client from reading properties belonging to another org. Both mechanisms exist in parallel, for different reasons.

---

3. Stripe subscription status driving feature access

Two concepts must never collapse into one: **billing state** (what Stripe says — active, trialing, past_due, canceled, unpaid) and **entitlements** (what the organization can actually do — originate new loans, import in bulk, access the API, number of seats, audit log retention).

The subscriptions table mirrors Stripe's truth: org_id, Stripe subscription ID, plan ID, status, current period boundaries, trial end, cancel-at-period-end flag, and a grace_period_ends_at column that exists independently of Stripe's status. The plans table stores the entitlement matrix as a structured JSON blob (feature flags, numeric limits, seat counts) plus the Stripe price ID it maps to.

Entitlements are derived, never stored per-organization. A single stored function takes an org_id and returns the effective entitlements by joining subscription → plan and applying business rules: active and trialing get full features, past_due within grace period gets full features, past_due beyond grace gets degraded access, canceled/unpaid gets read-only with a 30-day export window. That function is called from two places: RLS policies on gated tables (new loan origination, bulk import, API access), and the frontend state cache.

Enforcement lives at both layers. The frontend gate is UX — do not show buttons for features the user cannot use. The RLS gate is security — a user who crafts a POST with the right shape still gets denied at the database layer. Frontend-only enforcement is trivially bypassed and is not enforcement.

Stripe webhook handling is idempotent and signature-verified. Every incoming webhook has its event_id logged before processing. If the same event_id arrives again (Stripe retries aggressively), the handler returns 200 immediately without re-running the mutation. The webhook signature is verified on every request using Stripe's library; requests with invalid signatures never touch the database.

Grace periods are a property of the application, not Stripe. Stripe moves a subscription to past_due on the first failed invoice. The application decides whether that triggers immediate access loss, a 3-day soft-lock, or a 30-day read-only window. The subscription row's grace_period_ends_at column is set by the invoice.payment_failed handler and cleared by invoice.payment_succeeded. The entitlement function checks whether the grace period has expired — not raw Stripe status — when deciding to downgrade features.

---

4. Loan amortization at scale

The naive approach materializes every scheduled payment as a row at loan origination. 100,000 30-year mortgages equals 36 million rows. Every rate change on an ARM rewrites thousands of rows. Every extra principal payment shifts the whole tail. This approach collapses somewhere between a few thousand and ten thousand loans depending on how many modifications the product supports.

The sane approach treats the amortization schedule as a **pure function** of the loan's current terms and remaining balance, and materializes only what is needed when it is needed.

A loan_terms table stores immutable snapshots. Every origination writes one row (effective from origination date). Every modification (rate change, term extension, modification, recast) writes another row with the new effective date. The loan's current terms at any historical moment equals the most recent terms row with effective_date on or before that moment. This pattern also makes regulatory reporting straightforward because the state-at-a-point-in-time is always recoverable.

Balance is derived from the ledger, not stored as truth. The loan_transactions table is append-only: disbursements are positive, payments split into principal/interest/fee/escrow are negative, adjustments and reversals are new rows rather than mutations. Balance at any moment equals the sum of ledger entries on or before that moment. A denormalized current_balance_cents on the loans row is maintained by an AFTER INSERT trigger for query speed, and a nightly reconciliation job recomputes it from the ledger and alerts on any drift greater than one cent.

A stored function computes the amortization schedule on demand given a loan_id and a date range. It reads the ledger to get the balance at the range start, reads the effective terms, and walks month by month projecting interest and principal splits. For a UI display of a single loan's upcoming 12 months, this runs in under 50ms. For portfolio-wide reporting, the function is not called per loan — the system pre-materializes what is needed into a snapshot table.

Declarative partitioning on the ledger by month keeps index bloat manageable past 100 million rows. Reads for a single loan's last 12 months touch at most 12 partitions. Old partitions can be moved to cheaper storage or archived to S3-compatible storage once they are outside the retention window.

Pre-materialize narrow snapshots, compute wide schedules on demand. End-of-month balance per loan in a month_end_balances table means the chief credit officer's portfolio report runs in seconds instead of minutes. A 30-year amortization schedule for a UI detail page materializes nothing — it is a function call.

---

5. Audit logging

A fintech audit log answers three questions: "who did what to which record when," "what was the state of the record before and after," and "is the log itself trustworthy." Supabase's native tooling covers the first two; the third is where the architecture matters.

Database-level change capture via triggers. Every tenant table has an audit trigger that fires after any INSERT, UPDATE, or DELETE and writes a row to an audit_log table. The row captures: org_id, table name, row primary key, operation type, before-state JSON, after-state JSON, actor (Auth UUID at the moment of the write), actor's role, client IP from the request headers, request ID for correlation, and a cryptographic hash that includes the prior row's hash. The hash chain means any retroactive tampering with the log is detectable — changing a middle row invalidates every hash after it.

Separation of duties. The audit_log table has RLS policies that permit SELECT for auditors (a specific role or org-level permission) and INSERT only from the trigger function. No user — not even the application service_role — can UPDATE or DELETE from audit_log. The trigger runs as a SECURITY DEFINER function owned by a restricted role.

Application-level event log for business events. Not everything that matters is a row change. "User attempted to originate a loan but was blocked by entitlement check" is an event that matters to compliance but produces no database mutation. A separate events table captures these: event_type, org_id, actor, payload, timestamp. Written from the API layer, not the frontend.

Retention matches the regulatory regime. For a mortgage servicer subject to RESPA, that is typically 7 years post-payoff. For a broker under state licensing regimes, it is often longer. The audit_log partitioning strategy mirrors this — monthly partitions, automatically archived to cold storage after 90 days, purged only after the regulatory retention window plus a buffer.

External replication for true WORM. For the highest-stakes environments, audit entries are streamed to an append-only external store (S3 with Object Lock, or a dedicated audit log service like AWS CloudTrail, Datadog, or a SIEM) within seconds of being written. The in-database log is for querying; the external log is for regulatory attestation that the database itself could not have been tampered with.

---

6. Regulatory reporting patterns

Fintech regulatory reporting has two characteristics that drive the architecture: **reports are run retrospectively against state at a specific date**, and **the same report must produce identical output if run twice**. Both demand point-in-time queryability.

Temporal columns everywhere that matter. Loans, payments, and loan_terms all carry an effective_date (when the change is considered to have happened in business time) that is separate from the created_at/updated_at timestamps (when the row was written). A loan modification effective January 15 but entered on February 3 shows up in the January 15 state snapshot when running a March report — not in the February 3 snapshot.

Immutable snapshots at reporting boundaries. End of month, end of quarter, end of year: a snapshot job runs at T+1 day and writes a row per loan to a snapshot table with every field the regulator cares about (balance, delinquency status, days past due, accrued interest, modifications in the period). That snapshot is the official answer for that period — the live ledger continues to accept late-posting entries, but the snapshot is frozen.

Report definitions are versioned code, not ad-hoc queries. A report like “HMDA LAR filing” or “Call Report Schedule RC-K” is defined as a stored function with a version number, checked into the repository, and run against snapshots. Re-running last year’s Q3 report against this year’s code produces today’s interpretation of last year’s data; re-running it against the versioned code from the time of filing produces the original filing. Both are needed for regulator conversations.

Differences between accounting and regulatory are first-class. GAAP interest recognition on a non-accrual loan is different from how it is reported to the CFPB. The loan tables carry the raw events; derived views compute both angles. Never collapse them — auditors will ask for both.

Reconciliation between Stripe, the ledger, and what the borrower sees. At month-end, the sum of all Stripe charges in the period must equal the sum of all payment rows in succeeded status must equal the sum of ledger entries sourced from Stripe. A mismatch of even a few cents is a compliance finding. A nightly reconciliation job performs this three-way reconciliation and alerts on drift.

---

7. Multi-currency

If the product ever handles more than one currency — loans originated in CAD while the servicer reports in USD, a borrower paying EUR on a GBP loan, or an FX-denominated portfolio — currency cannot be an afterthought.

Every money column carries its own currency column, always. There is no “the USD column” — there is amount_cents and currency_code (ISO 4217, 3 characters). A loan in Canadian dollars and a loan in Australian dollars never share a “balance” column without accompanying “in what currency.”

Storage in minor units, not major. 150.50 USD is stored as 15050 cents. 1234.567 BHD (Bahraini dinar, 3 decimal places) is stored as 1234567. A separate currency metadata table gives the decimal places per ISO code. Display layer does the formatting; the database does the math on integers only.

FX rates are their own domain. A fx_rates table captures rate, source, effective timestamp, and reference period. Payments made in a currency other than the loan currency are converted using the rate in effect at the payment's effective date — not at posting date. Two separate ledger entries post: the payment in the borrower's currency, then a conversion entry into the loan's currency, with the rate and source recorded.

Reporting currency is a parameter. The same loan portfolio can be reported in USD for the board, CAD for the Canadian regulator, and EUR for a European investor. Reports take a reporting_currency parameter and convert on the fly using period-end rates.

Rounding rules are explicit and consistent. Banker's rounding (round-half-to-even), truncation, and round-half-away-from-zero all produce different totals over millions of rows. Pick one, document it, use the same rounding function everywhere. Inconsistency here is where the cent-level reconciliation failures originate.

---

8. ACH vs. card reconciliation

Card payments and ACH payments have fundamentally different timing and failure semantics, and treating them the same is a reconciliation disaster waiting to happen.

Card payments are near-synchronous and rarely reverse. A Stripe card charge settles in 2 business days. Chargebacks exist but are rare and typically happen within 60 days. When `payment_intent.succeeded` fires, the money is effectively yours — exceptions are disputes that flow through a separate reversal path months later.

ACH is asynchronous and reverses for up to 60 days. A Stripe ACH debit fires `payment_intent.processing` at submission, `payment_intent.succeeded` typically 4 business days later, and then can fail via `payment_intent.payment_failed` up to 60 days after that (NSF, unauthorized, account closed). For consumer Reg E claims, the window is 60 days. For commercial, it can be longer.

The payment row has a status lifecycle that mirrors this. `initiated` → `processing` → `succeeded` → potentially `reversed`. Ledger entries are gated: principal/interest allocation ledger rows are not posted until the payment is `succeeded`. If the payment later reverses, the reversal is written as new negative-amount ledger rows with a clear source reference, not by deleting the original.

Reconciliation files from the processor are the source of truth. Stripe's balance transaction ID, not the internal payment ID, is the anchor for the month-end reconciliation. Every ledger row sourced from a card or ACH event carries the Stripe balance transaction ID; the nightly reconciliation matches the internal ledger to Stripe's balance transactions and surfaces any unmatched entries on either side for human investigation.

Pending authorizations are not revenue. For card payments, the timing difference between authorization and capture is usually negligible. For ACH, the 4-day gap means money shown as

“received” in the UI must be distinguishable from money that has cleared. A funds_status column on the payment (pending, clearing, cleared, returned) reflects this; the dashboard shows cleared funds separately from in-flight funds.

Returns are their own entity. ACH returns arrive via a webhook typically 1-5 business days after the original “success” notification. They carry a return code (R01 insufficient funds, R08 payment stopped, R10 unauthorized, etc.) that drives downstream behavior — an R10 unauthorized return often triggers a compliance review. Returns are modeled as first-class rows with a foreign key back to the original payment, not as mutations of the original payment row.

---

9. RLS CI test harness

The scariest failure mode in a Supabase multi-tenant application is a missing RLS policy — a table added in a migration that did not enable RLS and shipped to production. That table is readable by every authenticated user in every organization. In fintech that is a breach, not a bug.

The CI test harness catches this before merge. A test script runs on every pull request and enumerates every table in the public schema. For each, it asserts: row-level security is enabled; at least one policy exists for SELECT, INSERT, and UPDATE; the table has an org_id column (unless explicitly allowlisted); and every policy references the current_org_id() helper. Any violation fails the build.

Positive tests per policy. For each tenant table, a test seeds data in two different organizations, signs a JWT as a user in org A, and asserts: the user can see the user’s org rows, cannot see org B’s rows, cannot insert a row with org B’s org_id, cannot update a row to change its org_id to org B, cannot bypass RLS by setting current_org_id via any path. These tests run against a real Postgres with RLS active — not mocked.

Negative tests per role. A viewer cannot originate a loan. A loan_officer cannot change a user’s role. A user with a removed membership cannot read anything in the organization. Each of these is a direct SQL assertion against the policies.

Service role tests are separate. The webhook handler and admin jobs bypass RLS. Tests for those code paths assert that the SQL explicitly scopes by org_id — that a webhook handler for org A cannot accidentally mutate data in org B, even though RLS is not protecting it.

Schema governance as a separate check. A second CI job enforces architectural invariants: no new table lacks an org_id column, no existing table drops RLS, no policy references the Auth UUID directly when current_org_id() should be used, no GRANT statement opens permissions on audit tables to non-auditor roles.

---

10. Securing Supabase for a fintech workload

A fintech Supabase deployment needs a security posture that goes well beyond the defaults.

Key management. The anon key is in the browser and is assumed public — all security flows through RLS and JWT claims. The service_role key is the database god-mode credential and lives in exactly three places: the server-side environment variables of backend services, a password manager, and the recovery escrow. It is never committed to git, never pasted in Slack, never put into a client bundle. A leaked service_role key is equivalent to a database compromise. Rotate on any suspicion of leak, rotate on employee departure, rotate annually.

JWT secret rotation. Supabase JWTs are signed with a project-level JWT secret. Rotating it invalidates every active session. Plan the rotation for an announced maintenance window, rotate, force all clients to re-authenticate. Put this on a 90-day cadence as a matter of hygiene, not just for breach response.

SECURITY DEFINER is a loaded gun. Postgres functions declared SECURITY DEFINER run with the privileges of the function owner, bypassing caller's RLS. They are necessary for some flows (the custom JWT hook, the entitlements function, the audit log trigger). Every single one must: be owned by a minimal-privilege role, have search_path set explicitly in the function body (prevents search-path-based privilege escalation), and be audited during code review. A SECURITY DEFINER function that accepts a user-controlled table or column name as input is a SQL injection vector even if the function body uses parameterized queries.

Network isolation where possible. Supabase's managed offering exposes the Postgres port through Supabase's edge. Direct database connections should use the transaction pooler, not the direct connection, for application workloads. If the regulatory regime allows (and the cost permits), put Supabase in a dedicated VPC with private networking to the application servers — otherwise, every Postgres connection traverses the public internet encrypted by TLS, which is fine for most SaaS but worth considering for high-stakes fintech.

Rate limiting at the edge. Supabase Auth endpoints (signup, login, password reset) are favorite brute-force targets. Put Cloudflare or Railway's edge in front with rate limits: sign-up by IP (low), login attempts by email (very low, to trigger lockout), password reset by email (low), general API requests (moderate). Abuse of the password reset flow is a common precursor to credential stuffing.

MFA required for admin roles. Any user with admin or owner role must have TOTP or WebAuthn enrolled. This is enforced at login — a user with the admin role and no MFA method gets denied access until they enroll. Supabase does not do this by default; the application enforces it via a post-login check that blocks navigation to admin routes.

Session management. Refresh tokens rotate on every use. Session inactivity times out at 30 minutes for admin roles, 24 hours for standard users. Long-lived sessions ("remember me") are explicitly disabled for admin roles. Every session is tied to a device fingerprint; detecting a session being used from a new IP block triggers a re-authentication challenge.

PII encryption at rest, above and beyond disk encryption. Social security numbers, bank account numbers, tax IDs, and similar sensitive columns are encrypted at the application layer before being

stored. Keys come from a managed KMS (AWS KMS, GCP KMS, HashiCorp Vault) — never from application config. The ciphertext is what is in the database; decryption happens in the application only when displaying to an authorized user. A compromised database dump yields ciphertext, not SSNs.

Backup encryption and access control. Supabase's automated backups are encrypted, but the organization's backup retrieval flow needs its own access controls — a backup download is a full database dump with plaintext of everything not application-encrypted. Limit who can request a backup restore. Log every backup access.

Webhook signature verification. Every incoming webhook (Stripe, ACH processor, KYC vendor, document signing service) is verified using the provider's signature before any processing. Unsigned or invalid-signature requests never touch the database. Webhook secrets rotate on a schedule and on any suspected exposure.

Database-level audit logging. Enable pg_audit (or equivalent) for DDL statements, role changes, and connections from privileged accounts. This is separate from the application-level audit_log table — it is the Postgres-level record of operational actions. Pipe to a SIEM.

Dependency surface. Every npm package in the stack is a potential supply-chain attack vector (the recent history has several high-profile examples). Dependabot on high-priority severity, a dependency review in every PR, lockfile integrity checks in CI, and a well-practiced procedure for emergency rotation if a popular dependency is compromised.

Breach response plan that is tested. Write the runbook before it is needed: which keys to rotate, which sessions to invalidate, who to notify (regulators have hard breach-notification timelines — GLBA safeguards rule requires notification within 30 days, state regulations vary down to 72 hours), which data lineage questions need to be answerable ("what data was accessed between X and Y"). Run a tabletop drill annually. Untested runbooks fail on the day they are needed.

Compliance posture. SOC 2 Type II is table stakes for fintech sales. Add PCI-DSS if touching card data (even if Stripe handles the card data, the system interfaces with it). GLBA Safeguards Rule applies to any U.S. financial institution. State-level privacy regimes (CCPA and similar) add requirements. Supabase's own compliance (they hold SOC 2) rolls up into the application's, but it does not replace the attestation needed for the service itself.

Monitoring and alerting, specifically tuned for fintech threats. Alerts on: failed-login spikes (credential stuffing), sudden query volume from a new IP (data exfiltration), any SELECT on audit_log from a non-auditor role, any UPDATE on audit_log at all, any connection using the service_role key from an unexpected source, any new SECURITY DEFINER function appearing in the schema, and any table having row level security disabled.

---

11. The short list of things that bite you later

- Money columns that are not integer cents — or integer minor units with currency code alongside
- RLS without membership status check (ghost sessions after user removal)
- FKs without RLS, or RLS without FKs — both matter for different reasons
- Tenancy derived from user instead of from an explicit membership join
- Webhook handlers that are not idempotent and signature-verified
- Amortization pre-materialized at origination with no plan for modifications
- Stripe status checked in the frontend but not in RLS — trivially bypassed
- No audit log table, or an audit log living in the same database without tamper-evidence
- Using the Auth UUID in policies when the semantic is tenancy, not identity
- SECURITY DEFINER functions without an explicit search_path (privilege escalation)
- Single currency assumption baked in, then multi-currency requested 18 months later
- ACH treated like card payments — reversal model entirely missing
- Regulatory reports as ad-hoc queries rather than versioned code against snapshots
- No RLS CI test harness — the day a table ships without a policy is the day of a breach
- Service_role key used in any client-reachable code path
- PII encrypted only by disk encryption, not field-level above the database